

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



AAT BDA PROJECT REPORT on

Urban Ride-Sharing Fleet Analytics Using Apache Cassandra

Submitted by

Prathith Suresh Rao	(1BM23CS247)
S Gaja Lakshmi	(1BM23CS278)
S Nagashree	(1BM23CS280)
Saanvi S	(1BM23CS282)

Under the Guidance of
Prof. LEELAVATHI.B
Assistant Professor, BMSCE

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B. M. S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
FEB-2026 to MAY-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the project work entitled “**PROJECT TITLE**” carried out by **Prathith Suresh Rao (1BM23CS247), S Gaja Lakshmi (1BM23CS278), S Nagashree (1BM23CS280) AND Saanvi S (1BM23CS282)** who are Bonafide students of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The AAT report has been approved as it satisfies the academic requirements in respect of Big Data Analytics (**23CS6PCBDA**) work prescribed for the said degree.

Signature

Prathith Suresh Rao	(1BM23CS247)
S Gaja Lakshmi	(1BM23CS278)
S Nagashree	(1BM23CS280)
Saanvi S	(1BM23CS282)

Signature of the Guide

Prof. Leelavathi. B
Assistant Professor
BMSCE, Bengaluru

TABLE OF CONTENT

Sl.NO	TITLE	Page No.
1	INTRODUCTION	1
2	DATASET COLLECTION AND INSPECTION	2
2.1	Dataset Generation	2
2.2	Data Entities	2
2.3	Initial Data Inspection	2
3	BIG DATA ANALYSIS	3
3.1	Keyspace And Table Design	3
3.2	Unlogged Batch Ingestion	3
3.3	Auto-Expiring Surge Pricing	3
3.4	Concurrent Driver Statistics	4
3.5	Dynamic Vehicle Amenities	4
3.5	Data Archival Via Csv Export	5
4	EXPLORING DATA DISTRIBUTION AND ANALYSIS	6
4.1	High-Value Ride Targeting	6
4.2	Commuter Exhaust Analysis	7
4.3	Fleet Capabilities Segmenting	7
4.4	Driver Performance and Power-Law Distribution	8
4.5	Time-Series Burst Analysis	9
5	CONCLUSION	10

1. Introduction

This project explores how a wide-column NoSQL database can power the telemetry infrastructure behind a modern mobility and ride-sharing platform (e.g., Uber, Ola, RapidRide). Ride-sharing applications generate continuous, high-velocity data: GPS location pings, ride completion logs, and dynamic pricing metrics. Relational databases often struggle to scale with this volume of time-series data.

Apache Cassandra was selected for this architecture because of its masterless, peer-to-peer design, which ensures high availability with no single point of failure. The project aims to demonstrate Cassandra's ability to handle massive write throughput, auto-expire temporary data structures (TTL), and manage concurrent updates to driver metrics seamlessly.

Key Objectives:

- Design a time-series schema partitioned by city zones for highly optimised regional analytics.
- Track driver lifetime ride counts using CRDT-based atomic counters.
- Implement auto-expiring "Surge Pricing" using Cassandra's native Time-To-Live (TTL) feature.
- Map dynamic vehicle features (e.g., WiFi, AC) using List collections to avoid expensive joins.
- Demonstrate data portability via CSV export and import.

2. Data Collection and Inspection

2.1 Dataset Generation A realistic synthetic dataset was generated using a custom Python script to simulate a high-traffic urban environment across 8 major zones (e.g., Koramangala, Indiranagar, Whitefield). The resulting CQL dataset successfully populated the `FleetDB` keyspace with 50 distinct vehicles and 500 completed ride records.

2.2 Data Entities

- **Vehicles (50 rows):** Stores driver profiles and vehicle specifications.
- **Completed_Rides (500 rows):** Time-series log of all completed trips, including distance and fare.
- **Driver_Stats (50 rows):** Real-time aggregation of lifetime rides per driver.
- **Surge_Pricing (4 rows):** Temporary fare multipliers activated by external triggers (e.g., Heavy Rain).

2.3 Initial Data Inspection Following the ingestion of the dataset via the Cassandra Shell (`cqlsh`), a preliminary inspection was conducted to verify data integrity.

```
cqlsh> SELECT COUNT(*) FROM fleetdb.completed_rides;

count
-----
    500
(1 rows)
```

```
cqlsh> SELECT COUNT(*) FROM fleetdb.vehicles;

count
-----
    50
(1 rows)
```

Verification of the 500 generated rides and 50 registered fleet vehicles.

3. Big Data Application

The RapidRide system leverages several advanced features native to Apache Cassandra to meet its high-throughput operational requirements.

3.1 Keyspace and Table Design

The `FleetDB` keyspace was initialised with a `SimpleStrategy` replication factor. The core analytical table, `Completed_Rides`, utilises a composite primary key structure: `PRIMARY KEY (zone_id, ride_time, ride_id) WITH CLUSTERING ORDER BY (ride_time DESC)`. Partitioning by `zone_id` ensures that all rides for a specific geographic area are stored contiguously on disk, while clustering by `ride_time` allows the system to fetch the most recent rides with $O(1)$ partition seek time.

3.2 Unlogged Batch Ingestion

To simulate rapid data dumps (e.g., a batch of ride telemetry arriving simultaneously from edge servers), ride data was grouped contextually and ingested using `BEGIN UNLOGGED BATCH`. While logged batches are used for atomic consistency across partitions, unlogged batches provide the necessary write-speed overhead required for high-velocity IoT telemetry ingestion.

3.3 Auto-Expiring Surge Pricing (TTL)

Surge pricing is inherently temporary. Rather than relying on background application jobs to delete old surge multipliers, the system utilises Cassandra's Time-To-Live feature (`USING TTL 3600`). This ensures server-side, automatic expiration of data.

```
cqlsh> SELECT zone_id, surge_multiplier, active_reason, TTL(surge_multiplier) AS seconds_remaining
... FROM fleetdb.surge_pricing;
```

zone_id	surge_multiplier	active_reason	seconds_remaining
Whitefield	1.3	Traffic Snarl	1054
Indiranagar	1.5	Concert Event	7054
Electronic_City	2.1	Office Rush Hour	1654
Koramangala	1.8	Heavy Rain	3454

(4 rows)

```
cqlsh> SELECT zone_id, surge_multiplier, active_reason, TTL(surge_multiplier) AS seconds_remaining
... FROM fleetdb.surge_pricing;
```

zone_id	surge_multiplier	active_reason	seconds_remaining
Whitefield	1.3	Traffic Snarl	1044
Indiranagar	1.5	Concert Event	7044
Electronic_City	2.1	Office Rush Hour	1644
Koramangala	1.8	Heavy Rain	3444

Live countdown of active surge pricing multipliers demonstrating the TTL feature.

3.4 Concurrent Driver Statistics (Counters)

As drivers complete rides concurrently, updating their lifetime stats in a traditional database requires expensive read-modify-write locks. By defining `total_rides` as a `COUNTER` column, the database utilises Conflict-Free Replicated Data Types (CRDTs) to guarantee accurate increments without blocking writes.

```
cqlsh> SELECT * FROM fleetdb.driver_stats LIMIT 10;
```

vehicle_id	total_rides
c41c5077-7b29-4e08-90bf-2ea3ed7f3c1b	6
d4abeff2-3666-4d69-8577-8426d9db52bd	9
b646d1c6-30e5-4af1-83d4-72ccd547f0d2	16
966aef9c-d73b-4ba2-bd7d-bf25b4740f06	14
c85baf67-2b7d-4936-94bc-90cb769ddb94	8
bc653787-3c4a-4e87-bdd7-a9dcc80aaa4f	8
c63ff44b-b155-4974-bc8f-900353d0a68b	13
1dd8f577-aceb-43ae-987c-639f6d7cc21b	14
b88f7d48-b3c1-43d0-9d70-232e47af45b8	9
2fa22b9a-1a01-44c7-8e98-384d2664a863	8

(10 rows)

Atomic counters tracking the total number of rides completed by each vehicle.

3.5 Dynamic Vehicle Amenities (Collections)

Vehicle configurations change frequently (e.g., adding a Dashcam or WiFi). Instead of heavily normalised relational tables requiring `JOIN` operations, vehicle amenities are stored natively as a `LIST<TEXT>` collection.

```
cqlsh> SELECT driver_name, car_model, amenities
... FROM fleetdb.vehicles
... LIMIT 5;
```

driver_name	car_model	amenities
Siddharth Kumar	Toyota Innova	['Electric', 'Dashcam', 'Hatchback', 'Top_Rated']
Priya Gowda	Toyota Innova	['AC', 'Electric', 'Sedan']
Vikram Iyer	Honda City	['Electric', 'Dashcam']
Priya Menon	Kia Seltos	['Electric', 'WiFi', 'Sedan', 'Dashcam']
Pooja Rao	Maruti Dzire	['AC', 'SUV', 'Dashcam', 'Top_Rated']

(5 rows)

Utilisation of List collections to store dynamic arrays of vehicle features.

3.6 Data Archival via CSV Export

To integrate the operational Cassandra database with downstream analytical tools (e.g., Hadoop or Spark data lakes), data portability was validated using the native `COPY TO` and `COPY FROM` commands.

```
cqlsh> COPY fleetdb.completed_rides TO '/tmp/completed_rides.csv' WITH HEADER = TRUE;
HEADER = TRUE;Using 11 child processes

Starting copy of fleetdb.completed_rides with columns [zone_id, ride_time, ride_id, distance_km, fare_amount, vehicle_id].
Processed: 500 rows; Rate: 4624 rows/s; Avg. rate: 4624 rows/s
500 rows exported to 1 files in 0.175 seconds.
cqlsh> COPY fleetdb.archived_rides FROM '/tmp/completed_rides.csv' WITH HEADER = TRUE;
Using 11 child processes

Starting copy of fleetdb.archived_rides with columns [zone_id, ride_time, ride_id, distance_km, fare_amount, vehicle_id].
Processed: 500 rows; Rate: 776 rows/s; Avg. rate: 1170 rows/s
500 rows imported from 1 files in 0.427 seconds (0 skipped).
cqlsh> SELECT COUNT(*) FROM fleetdb.archived_rides;

count
-----
500
(1 rows)
```

Successful export of the completed rides to CSV and subsequent re-import into the archival table.

4. Exploring Data Distribution and Analysis

To extract business intelligence from the RapidRide cluster, direct CQL queries were executed targeting specific partitions and features.

4.1 High-Value Ride Targeting

To optimise driver allocation, the system must identify zones generating premium fares. By filtering for fares exceeding ₹400 (`WHERE fare_amount > 400`), the cluster returned 50 specific high-value rides.

```
cqlsh> SELECT zone_id, ride_time, distance_km, fare_amount
... FROM fleetdb.completed_rides
... WHERE fare_amount > 400
... ALLOW FILTERING;
```

zone_id	ride_time	distance_km	fare_amount
Jayanagar	2026-05-13 09:36:45.000000+0000	24.6	425.37
Jayanagar	2026-05-13 08:53:22.000000+0000	24.7	440.02
Jayanagar	2026-05-13 06:26:34.000000+0000	22.8	405.81
Jayanagar	2026-05-12 20:28:01.000000+0000	23.5	420.09
Jayanagar	2026-05-12 17:16:47.000000+0000	22.9	405.01
Jayanagar	2026-05-11 20:57:40.000000+0000	24.9	432.85
Jayanagar	2026-05-11 11:03:10.000000+0000	23.0	410.57
Jayanagar	2026-05-10 20:56:01.000000+0000	23.7	420.09
Whitefield	2026-05-13 01:12:42.000000+0000	23.3	417.03
Whitefield	2026-05-12 04:29:19.000000+0000	24.4	418.26
Whitefield	2026-05-12 01:47:46.000000+0000	23.7	407.01
Whitefield	2026-05-11 20:23:48.000000+0000	22.9	408.18
Whitefield	2026-05-10 14:07:31.000000+0000	24.1	419.82
Indiranagar	2026-05-13 14:25:01.000000+0000	23.9	418.93
Indiranagar	2026-05-12 19:58:26.000000+0000	22.3	400.73
Indiranagar	2026-05-11 16:25:38.000000+0000	24.9	438.05
Indiranagar	2026-05-11 15:35:09.000000+0000	23.4	403.78
Indiranagar	2026-05-10 10:20:23.000000+0000	24.7	427.06
Marathahalli	2026-05-13 14:06:30.000000+0000	24.8	431.68
Marathahalli	2026-05-12 22:29:52.000000+0000	23.4	419.23
Marathahalli	2026-05-12 03:40:54.000000+0000	22.7	401.23
Marathahalli	2026-05-11 23:50:43.000000+0000	24.8	431.42
Marathahalli	2026-05-10 13:57:05.000000+0000	23.2	401.62
HSR_Layout	2026-05-13 23:22:09.000000+0000	22.5	401.1
HSR_Layout	2026-05-13 11:15:15.000000+0000	23.2	409.36
HSR_Layout	2026-05-13 00:06:29.000000+0000	23.2	402.91
HSR_Layout	2026-05-12 15:47:33.000000+0000	23.2	408.19
HSR_Layout	2026-05-12 07:01:16.000000+0000	23.0	405.78
HSR_Layout	2026-05-11 22:35:42.000000+0000	23.8	416.71
HSR_Layout	2026-05-10 11:36:32.000000+0000	24.6	419.16
HSR_Layout	2026-05-10 08:59:53.000000+0000	24.7	438.34
Electronic_City	2026-05-13 10:50:17.000000+0000	22.5	402.06
Electronic_City	2026-05-13 02:28:17.000000+0000	24.6	433.39
Electronic_City	2026-05-12 01:08:53.000000+0000	23.3	401.12
Electronic_City	2026-05-11 21:27:14.000000+0000	24.2	432.68
Electronic_City	2026-05-11 17:49:48.000000+0000	22.4	405.73
Malleswaram	2026-05-13 20:36:59.000000+0000	22.6	402.7
Malleswaram	2026-05-12 17:14:45.000000+0000	22.2	401.99
Malleswaram	2026-05-12 15:24:48.000000+0000	24.3	431.17
Malleswaram	2026-05-12 00:27:29.000000+0000	23.2	404.68
Malleswaram	2026-05-11 18:20:28.000000+0000	24.3	415.77
Malleswaram	2026-05-11 04:34:42.000000+0000	25.0	439.47
Malleswaram	2026-05-10 17:30:54.000000+0000	23.2	410.62
Malleswaram	2026-05-10 11:09:04.000000+0000	23.3	418.01
Koramangala	2026-05-13 19:53:55.000000+0000	24.6	423.04
Koramangala	2026-05-13 07:59:38.000000+0000	22.6	406.0
Koramangala	2026-05-12 06:35:15.000000+0000	24.8	423.21
Koramangala	2026-05-12 04:35:41.000000+0000	23.5	421.12
Koramangala	2026-05-11 22:31:06.000000+0000	23.9	425.02
Koramangala	2026-05-11 06:17:36.000000+0000	23.7	406.71

(50 rows)

- **Insight:** Zones like Jayanagar and Whitefield consistently generated fares over ₹410. In a production environment, this query logic is used to dynamically update heat-maps for drivers, routing idle fleet vehicles toward high-revenue clusters.

4.2 Commuter Exhaust Analysis (Electronic City)

Analyzing traffic flowing out of major tech hubs is critical for fleet balancing. A targeted query was executed against the `Electronic_City` partition to find extreme long-distance trips (> 20km).

```
cqlsh> SELECT ride_time, distance_km, fare_amount, vehicle_id
_id = 'Electronic_City' AND dist ... FROM fleetdb.completed_rides
ALLOW FILTERING; ... WHERE zone_id = 'Electronic_City' AND distance_km > 20.0
... ALLOW FILTERING;
```

ride_time	distance_km	fare_amount	vehicle_id
2026-05-13 15:13:41.000000+0000	21.0	380.85	ecc9040c-fde7-4d5f-acf9-c679c695cc44
2026-05-13 11:18:41.000000+0000	22.2	393.38	1dd8f577-aceb-43ae-987c-639f6d7cc21b
2026-05-13 10:50:17.000000+0000	22.5	402.06	8e147842-0baa-4a26-8ff5-80c622c53ed6
2026-05-13 03:51:11.000000+0000	20.8	368.08	fb7b6eb9-bc4a-466e-b230-6becf0f7b887
2026-05-13 02:28:17.000000+0000	24.6	433.39	ac4bce85-b4e6-4ad8-804b-1c4ccf020424
2026-05-12 01:08:53.000000+0000	23.3	401.12	2a71cc7e-640c-40b8-ba40-2180127d6ebc
2026-05-11 21:27:14.000000+0000	24.2	432.68	79791a27-95c2-4d59-990d-5a38950df068
2026-05-11 17:49:48.000000+0000	22.4	405.73	2fa22b9a-1a01-44c7-8e98-384d2664a863
2026-05-11 11:47:28.000000+0000	22.3	396.67	f3ee8780-c6a8-4d7c-8bba-dc6ab5232883
2026-05-11 09:55:18.000000+0000	20.6	362.41	1e2bb271-6c28-450f-8e5e-68c024cf3866
2026-05-10 13:28:18.000000+0000	21.2	384.86	b04e8a22-6754-4f5d-91c5-f787ed80f67c
2026-05-10 07:17:57.000000+0000	21.1	384.31	96151605-7372-4c4d-956f-0515e35e1c68

(12 rows)

- **Insight:** The query returned 12 specific trips, with distances peaking at 24.6km. The use of `ALLOW FILTERING` on a specific partition demonstrates Cassandra's ability to execute multi-variable queries (zone + distance constraints) efficiently without a full cluster scan.

4.3 Fleet Capability Segmenting (Collection Queries)

Unlike relational databases that require complex and computationally expensive `JOIN` operations to map vehicles to their features, this schema utilises native `LIST<TEXT>` collections.

```
cqlsh> SELECT driver_name, car_model, amenities
ALLOW FILTERIN ... FROM fleetdb.vehicles
... WHERE amenities CONTAINS 'WiFi'
... ALLOW FILTERING;
```

driver_name	car_model	amenities
Priya Menon	Kia Seltos	['Electric', 'WiFi', 'Sedan', 'Dashcam']
Vikram Iyer	Hyundai i20	['WiFi', 'Dashcam']
Rajesh Iyer	Maruti Dzire	['Dashcam', 'Hatchback', 'WiFi']
Siddharth Tiwari	Kia Seltos	['WiFi', 'Top_Rated', 'Premium_Audio']
Vikram Sharma	Toyota Innova	['Electric', 'WiFi', 'Premium_Audio']
Manoj Reddy	Mahindra XUV700	['AC', 'Top_Rated', 'WiFi', 'SUV']
Neha Singh	Hyundai i20	['SUV', 'WiFi']
Rajesh Sharma	Tata Nexon	['WiFi', 'Dashcam']
Amit Singh	Honda City	['WiFi', 'SUV', 'Top_Rated']
Karthik Reddy	Hyundai i20	['Hatchback', 'WiFi']
Aarav Gowda	Toyota Innova	['Dashcam', 'AC', 'WiFi', 'Hatchback']
Vihaan Gowda	Tata Nexon	['AC', 'Top_Rated', 'Sedan', 'WiFi']
Aarav Patel	Hyundai i20	['Dashcam', 'WiFi']
Amit DSouza	Maruti Dzire	['WiFi', 'Electric', 'SUV', 'Hatchback']
Suresh Reddy	Maruti Dzire	['Hatchback', 'WiFi']
Arjun Gowda	Hyundai i20	['Dashcam', 'Hatchback', 'WiFi']
Amit Tiwari	Maruti Dzire	['WiFi', 'Premium_Audio']
Vihaan Rao	Mahindra XUV700	['Premium_Audio', 'WiFi']

(18 rows)

- **Insight:** Using the `CONTAINS` operator, the system instantly identified 18 vehicles equipped with 'WiFi' (ideal for corporate clients). This proves NoSQL wide-column stores can handle dynamic inventory matching natively and with near-zero latency.

4.4 Driver Performance and Power-Law Distribution

A query filtering for highly active drivers (`WHERE total_rides > 12`) returned an elite tier of 11 drivers out of the 50 registered vehicles.

```
cqlsh> SELECT vehicle_id, total_rides
... FROM fleetdb.driver_stats
... WHERE total_rides > 12
... ALLOW FILTERING;
```

vehicle_id	total_rides
b646d1c6-30e5-4af1-83d4-72ccd547f0d2	16
966aef9c-d73b-4ba2-bd7d-bf25b4740f06	14
c63ff44b-b155-4974-bc8f-900353d0a68b	13
1dd8f577-aceb-43ae-987c-639f6d7cc21b	14
79791a27-95c2-4d59-990d-5a38950df068	13
1e2bb271-6c28-450f-8e5e-68c024cf3866	19
bd562a6d-ae0b-40c1-9af3-fcf9fabe7658	14
401d9a1a-37fb-4213-b87d-80fa50696c5e	14
9b6ea8fe-330f-45e7-a151-be087d872074	14
72c58d25-c7ec-455f-95a2-a199a29fbe5e	13
03b89299-797c-4360-a1ec-721a79ca4ab3	19

- **Insight:** Two specific vehicles completed a maximum of 19 rides each. Because `COUNTER` columns utilise CRDTs, the system successfully aggregated these 500 atomic increments across a distributed architecture without any read-lock bottlenecks.

4.5 Time-Series Burst Analysis

Cassandra's composite primary key design natively supports time-series data. By partitioning by `zone_id` and clustering by `ride_time DESC`, the system creates highly optimised data structures on disk.

```
cqlsh> SELECT ride_time, ride_id, distance_km, fare_amount
e >= '2026-05-12 ... FROM fleetdb.completed_rides
00'
AND ride_time <= '2026-05 ... WHERE zone_id = 'Koramangala'
... AND ride_time >= '2026-05-12T00:00:00.000+0000'
... AND ride_time <= '2026-05-12T23:59:59.000+0000';
```

ride_time	ride_id	distance_km	fare_amount
2026-05-12 23:17:22.000000+0000	b1640db0-48b9-11f1-a63d-6ff313661469	11.8	230.09
2026-05-12 22:45:00.000000+0000	b163988a-48b9-11f1-a63d-6ff313661469	5.4	145.74
2026-05-12 22:10:15.000000+0000	b1632350-48b9-11f1-a63d-6ff313661469	11.3	235.88
2026-05-12 21:01:45.000000+0000	b1626000-48b9-11f1-a63d-6ff313661469	18.7	339.9
2026-05-12 20:57:10.000000+0000	b16238f0-48b9-11f1-a63d-6ff313661469	9.1	205.07
2026-05-12 20:33:51.000000+0000	b161ead0-48b9-11f1-a63d-6ff313661469	16.7	309.57
2026-05-12 20:16:58.000000+0000	b1619cba-48b9-11f1-a63d-6ff313661469	10.1	212.74
2026-05-12 20:06:14.000000+0000	b1619cb0-48b9-11f1-a63d-6ff313661469	21.1	380.38
2026-05-12 19:31:12.000000+0000	b160d960-48b9-11f1-a63d-6ff313661469	5.7	138.93
2026-05-12 18:22:50.000000+0000	b1603d20-48b9-11f1-a63d-6ff313661469	7.8	173.75
2026-05-12 17:58:30.000000+0000	b15fef00-48b9-11f1-a63d-6ff313661469	14.2	271.73
2026-05-12 16:34:57.000000+0000	b15edd90-48b9-11f1-a63d-6ff313661469	16.5	305.37
2026-05-12 15:44:40.000000+0000	b15da510-48b9-11f1-a63d-6ff313661469	7.6	183.68
2026-05-12 15:13:59.000000+0000	b15cbab0-48b9-11f1-a63d-6ff313661469	15.0	280.24
2026-05-12 12:09:12.000000+0000	b159ad70-48b9-11f1-a63d-6ff313661469	7.2	166.3
2026-05-12 11:58:20.000000+0000	b1595f50-48b9-11f1-a63d-6ff313661469	7.8	167.23
2026-05-12 11:07:54.000000+0000	b15826d0-48b9-11f1-a63d-6ff313661469	1.7	77.58
2026-05-12 10:26:21.000000+0000	b157b1a0-48b9-11f1-a63d-6ff313661469	12.8	244.73
2026-05-12 09:36:43.000000+0000	b1571560-48b9-11f1-a63d-6ff313661469	19.6	344.1
2026-05-12 09:09:36.000000+0000	b1567920-48b9-11f1-a63d-6ff313661469	10.1	213.65
2026-05-12 08:03:21.000000+0000	b1558ec0-48b9-11f1-a63d-6ff313661469	12.1	234.53
2026-05-12 06:56:14.000000+0000	b1542f30-48b9-11f1-a63d-6ff313661469	8.6	184.22
2026-05-12 06:35:15.000000+0000	b153e110-48b9-11f1-a63d-6ff313661469	24.8	423.21
2026-05-12 06:21:18.000000+0000	b15392f0-48b9-11f1-a63d-6ff313661469	22.1	389.74
2026-05-12 04:45:47.000000+0000	b1523360-48b9-11f1-a63d-6ff313661469	18.5	330.83
2026-05-12 04:35:41.000000+0000	b1520c50-48b9-11f1-a63d-6ff313661469	23.5	421.12
2026-05-12 04:25:43.000000+0000	b151be3a-48b9-11f1-a63d-6ff313661469	8.9	184.35
2026-05-12 03:49:13.000000+0000	b1514900-48b9-11f1-a63d-6ff313661469	6.2	145.52
2026-05-12 00:59:18.000000+0000	b14f4d30-48b9-11f1-a63d-6ff313661469	5.2	145.12

(29 rows)

- **Insight:** Querying for a specific time window within the 'Koramangala' zone executes in $O(1)$ time complexity per partition. The database simply seeks to the partition and reads sequentially from the top of the LSM-tree, making it perfectly suited for real-time traffic burst analytics during rush hour windows.

5. Conclusion

This project successfully designed, populated, and analysed a complete Cassandra-backed telemetry architecture for an urban ride-sharing platform. The implementation of a realistic 500-ride synthetic dataset allowed for the comprehensive demonstration of core Cassandra mechanics.

Key Findings:

1. **Time-Series Superiority:** The composite primary key model proved highly effective for ride-sharing data. Partitioning by city zone and clustering by time allowed for instant extraction of traffic bursts without full-table scans.
2. **Operational Efficiency:** Features like TTL eliminated the need for application-level cleanup logic for surge pricing, while COUNTER data types handled high-concurrency driver statistics natively without conflict.
3. **Schema Flexibility:** The use of List collections allowed for the dynamic querying of vehicle capabilities (e.g., matching SUVs with WiFi) without the heavy performance penalty of relational table joins.

The analysis of the dataset revealed clear patterns in commuter exhaust out of tech hubs, power-law distributions in driver activity, and the precise identification of high-revenue clusters. Apache Cassandra's masterless architecture and purpose-built data structures make it an exceptionally powerful choice for high-velocity, geographically distributed telemetry workloads.